

Gurmat_work

July 2, 2025

```
[ ]: # Install necessary libraries

import pandas as pd
import numpy as np
import random
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder, LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

from deap import base, creator, tools, algorithms
import optuna
from sklearn.metrics import classification_report
import warnings
warnings.filterwarnings("ignore")

# Suppress Optuna's trial logs for cleaner output during optimization
optuna.logging.set_verbosity(optuna.logging.WARNING)

print("Libraries installed and imported successfully.")

[5]: # Load the dataset
# We'll try to load a pre-processed pickle file first for speed.
# If it doesn't exist, we load the CSV and create the pickle file.
try:
    df = pd.read_pickle('../dataPreprocessing/medical_data.pkl')
    print("Loaded data from medical_data.pkl")
except FileNotFoundError:
    print("Info: 'medical_data.pkl' not found. Attempting to load from CSV.")

# --- Data Cleaning and Preparation ---
```

```

# Replace '?' with NaN and handle potential errors
df.replace('?', np.nan, inplace=True)

# Define target and features
TARGET = 'readmitted_ind'
X = df.drop(TARGET, axis=1)
y = df[TARGET]

# Encode the target variable to be numeric (0 and 1)
le = LabelEncoder()
y = le.fit_transform(y)

# --- Feature Definition ---
# Define features to exclude based on your example
exclude_features = [
    'patient_nbr', 'encounter_id', 'diagnosis_tuple', 'readmitted', 'dummy'
]

# Filter out excluded columns that are not in the dataframe
existing_exclude_features = [col for col in exclude_features if col in X.
    ↪columns]
X = X.drop(columns=existing_exclude_features)

# Identify numeric and categorical features from the remaining columns
numeric_features = X.select_dtypes(include=np.number).columns.tolist()
object_features = X.select_dtypes(include=['object']).columns.tolist()

# --- Preprocessing Pipeline ---
# Create a column transformer for preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ('num', MinMaxScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore', drop='first',
    ↪sparse_output=False), object_features)
    ],
    remainder='passthrough'
)

# --- Train-Test Split ---
# Split data into a training set and a final hold-out test set
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nData loaded and preprocessed.")
print(f"Training set shape: {X_train.shape}")
print(f"Test set shape: {X_test.shape}")

```

```
print(f"Number of numeric features: {len(numeric_features)}")
print(f"Number of categorical features: {len(object_features)}")
```

Loaded data from medical_data.pkl

Data loaded and preprocessed.
 Training set shape: (62439, 183)
 Test set shape: (15610, 183)
 Number of numeric features: 173
 Number of categorical features: 10

```
[7]: device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.
    ↪mps.is_available() else "cpu")
    print(f"Using device: {device}")
```

Using device: mps

```
[8]: from sklearn.base import BaseEstimator, TransformerMixin

# --- Custom Transformer for Data Type Conversion ---
# This helper class ensures that all data passed to the OneHotEncoder is of
    ↪string type,
# preventing errors with mixed types (e.g., columns containing both numbers and
    ↪strings).
class TypeConverter(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self
    def transform(self, X, y=None):
        return X.astype(str)

# --- Genetic Algorithm Setup ---

# The number of features to select from
# We determine this by combining the numeric and object feature lists from the
    ↪previous cell.
N_FEATURES = len(numeric_features) + len(object_features)

# Define the fitness and individual creation for the GA
# We want to maximize accuracy, so weights=(1.0,)
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
# An individual is a list of bits (0 or 1) with a fitness attribute
creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox = base.Toolbox()

# Attribute generator: a gene is either 0 (not selected) or 1 (selected)
toolbox.register("attr_bool", random.randint, 0, 1)
```

```

# Individual initializer: creates an individual with N_FEATURES genes
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.
    ↳attr_bool, n=N_FEATURES)

# Population initializer
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# --- Fitness Evaluation Function ---
# This function determines how "good" a feature subset is by training a simple
    ↳model.
def evaluate_features(individual):
    # Get indices of selected features (where the gene is 1)
    selected_indices = [i for i, bit in enumerate(individual) if bit == 1]

    # If the GA creates an individual with no features, its fitness is 0.
    if not selected_indices:
        return 0.0,

    # Combine all feature names to create a master list
    all_feature_names = np.array(numeric_features + object_features)
    # Select the feature names based on the individual's genes
    selected_features = all_feature_names[selected_indices].tolist()

    # Separate the selected features back into numeric and categorical types
    sel_numeric = [f for f in selected_features if f in numeric_features]
    sel_categorical = [f for f in selected_features if f in object_features]

    # Create a nested pipeline for categorical features to first convert them
    ↳to strings
    categorical_transformer = Pipeline(steps=[
        ('tostring', TypeConverter()),
        ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first'))
    ])

    # Create a temporary pipeline with a preprocessor for ONLY the selected
    ↳features
    # This is crucial to evaluate the specific subset.
    pipeline = Pipeline([
        ('col_transformer', ColumnTransformer(
            transformers=[
                ('num', MinMaxScaler(), sel_numeric),
                ('cat', categorical_transformer, sel_categorical)
            ],
            remainder='drop' # Drop any columns that were not selected
        )),
        # Use a fast model like Logistic Regression for quick fitness evaluation
        ('classifier', LogisticRegression(random_state=42, max_iter=1000))
    ])

```

```

])

# For speed, we train and evaluate on a smaller sample of the data.
X_sample, _, y_sample, _ = train_test_split(X_train, y_train, test_size=0.
↪8, random_state=42, stratify=y_train)
pipeline.fit(X_sample, y_sample)
y_pred = pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# DEAP requires the fitness to be a tuple
return accuracy,

# --- Register GA Operators ---
toolbox.register("evaluate", evaluate_features)
toolbox.register("mate", tools.cxTwoPoint) # Crossover operator
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05) # Mutation operator
toolbox.register("select", tools.selTournament, tournsize=3) # Selection
↪operator

# --- Run the Genetic Algorithm ---
print("Starting Genetic Algorithm for feature selection...")
population = toolbox.population(n=50)
NGEN = 20 # Number of generations to run
CXPB, MUTPB = 0.5, 0.2 # Crossover and mutation probabilities

# Use a Hall of Fame to keep track of the single best individual found
hof = tools.HallOfFame(1)

# Set up statistics to track during evolution
stats = tools.Statistics(lambda ind: ind.fitness.values)
stats.register("avg", np.mean)
stats.register("std", np.std)
stats.register("min", np.min)
stats.register("max", np.max)

# Run the evolution process
algorithms.eaSimple(population, toolbox, cxpb=CXPB, mutpb=MUTPB, ngen=NGEN,
                    stats=stats, halloffame=hof, verbose=True)

# --- Get the Best Features from the Hall of Fame ---
best_individual = hof[0]
all_feature_names = np.array(numeric_features + object_features)
selected_features_mask = np.array(best_individual).astype(bool)
selected_features = all_feature_names[selected_features_mask].tolist()

print("\nGenetic Algorithm finished.")
print(f"Number of selected features: {len(selected_features)}")

```

```

print(f"Selected features: {selected_features}")

# --- Final Preprocessing with Selected Features ---
# Create the final preprocessor that will be used to prepare data for the CNN
sel_numeric_final = [f for f in selected_features if f in numeric_features]
sel_categorical_final = [f for f in selected_features if f in object_features]

# The categorical transformer pipeline to ensure data is string before one-hot
↳encoding
final_categorical_transformer = Pipeline(steps=[
    ('tostring', TypeConverter()),
    ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first',
↳sparse_output=False))
])

final_preprocessor = ColumnTransformer(
    transformers=[
        ('num', MinMaxScaler(), sel_numeric_final),
        ('cat', final_categorical_transformer, sel_categorical_final)
    ],
    remainder='drop' # IMPORTANT: Drop columns not selected by the GA to
↳prevent data type errors
)

# Transform the training and test sets using the final preprocessor
X_train_selected = final_preprocessor.fit_transform(X_train)
X_test_selected = final_preprocessor.transform(X_test)

# The number of input features for our CNN is now the number of columns in the
↳processed data
INPUT_SIZE = X_train_selected.shape[1]

print(f"\nData preprocessed with selected features. New input dimension for CNN:
↳ {INPUT_SIZE}")

```

Starting Genetic Algorithm for feature selection...

gen	nevals	avg	std	min	max
0	50	0.844319	0.0599553	0.709994	0.898334
1	30	0.887157	0.011548	0.828443	0.898334
2	33	0.894286	0.00507249	0.877514	0.900256
3	27	0.897277	0.00268344	0.880525	0.90032
4	31	0.898037	0.00309551	0.879628	0.901217
5	26	0.894689	0.0231606	0.748879	0.901217
6	41	0.900064	0.000981543	0.896413	0.901409
7	29	0.900805	0.000411624	0.899488	0.901409
8	24	0.900439	0.0032228	0.87886	0.901409
9	25	0.900682	0.00235888	0.884689	0.90173

10	28	0.900273	0.00448637	0.877258	0.90173
11	28	0.900268	0.00478137	0.876938	0.901922
12	24	0.898368	0.014623	0.822357	0.902306
13	32	0.900739	0.00401353	0.878091	0.902947
14	27	0.901171	0.00325813	0.878796	0.902562
15	35	0.901617	0.00251149	0.884433	0.902819
16	33	0.902137	0.000836611	0.897822	0.903011
17	35	0.901915	0.00245937	0.885586	0.903075
18	39	0.902583	0.000507089	0.900897	0.903075
19	32	0.901489	0.00511565	0.879693	0.903331
20	30	0.902322	0.00330581	0.879693	0.903331

Genetic Algorithm finished.

Number of selected features: 102

Selected features: ['time_in_hospital', 'num_lab_procedures', 'num_medications', 'number_inpatient', 'mb_admission_grp_2_ct', 'mb_discharge_grp_1_ct', 'mb_admission_type_ct', 'distinct_diag_count', 'diag_1_freq', 'diag_2_freq', 'diag_3_freq', 'LTIS_38_ind', 'LTIS_40_ind', 'LTIS_36_ind', 'LTIS_320_ind', 'LTIS_324_ind', 'CE_410_ind', 'CE_431_ind', 'CE_428_ind', 'CMN_155_ind', 'CMN_162_ind', 'CMN_191_ind', 'OF_570_ind', 'OF_584_ind', 'OF_277_ind', 'NBD_780_ind', 'NBD_852_ind', 'STI_861_ind', 'OCC_250_ind', 'OCC_989_ind', 'is_high_risk_combo', 'dx_428_ind_max', 'dx_707_ind_max', 'dx_707_ind_sum', 'dx_396_ind_sum', 'dx_440_ind_max', 'dx_453_ind_max', 'dx_571_ind_max', 'dx_571_ind_sum', 'dx_284_ind_max', 'dx_284_ind_sum', 'dx_482_ind_sum', 'dx_150_ind_sum', 'dx_282_ind_sum', 'dx_332_ind_max', 'dx_332_ind_sum', 'dx_719_ind_max', 'dx_423_ind_max', 'dx_423_ind_sum', 'dx_281_ind_max', 'dx_536_ind_max', 'dx_368_ind_max', 'dx_368_ind_sum', 'dx_515_ind_sum', 'dx_595_ind_max', 'dx_572_ind_max', 'dx_572_ind_sum', 'dx_681_ind_max', 'dx_681_ind_sum', 'dx_581_ind_max', 'dx_581_ind_sum', 'dx_537_ind_sum', 'dx_V46_ind_max', 'dx_519_ind_max', 'dx_300_ind_max', 'dx_300_ind_sum', 'dx_567_ind_max', 'dx_V49_ind_max', 'dx_V49_ind_sum', 'dx_094_ind_sum', 'dx_514_ind_max', 'dx_494_ind_sum', 'dx_042_ind_sum', 'dx_346_ind_max', 'dx_792_ind_max', 'dx_398_ind_max', 'dx_753_ind_max', 'dx_577_ind_max', 'dx_577_ind_sum', 'dx_444_ind_sum', 'dx_459_ind_sum', 'dx_337_ind_max', 'dx_337_ind_sum', 'dx_292_ind_sum', 'dx_V42_ind_max', 'dx_289_ind_sum', 'obesity_history_ind', 'mh_history_ind', 'encounter_ct', 'mb_readmitted_lt30_ct', 'mb_readmitted_gt30_ct', 'mb_readmitted_no_ct', 'mb_num_lab_procedures_ct', 'mb_num_medications_ct', 'mb_number_outpatient_ct', 'mb_number_emergency_ct', 'gender', 'change', 'diabetesMed', 'discharge_disposition', 'diagnosis_1', 'diagnosis_2']

Data preprocessed with selected features. New input dimension for CNN: 1397

```
[9]: class SingleShotCNN(nn.Module):
      def __init__(self, input_size, num_classes, num_filters, kernel_size,
↳ dropout_p):
          super(SingleShotCNN, self).__init__()
```

```

        # Convolutional layer
        self.conv1 = nn.Conv1d(in_channels=1, out_channels=num_filters,
        ↪kernel_size=kernel_size, padding='same')
        self.relu = nn.ReLU()
        # Global Max Pooling will reduce the dimension to (batch_size,
        ↪num_filters)
        self.pool = nn.AdaptiveMaxPool1d(1)
        self.dropout = nn.Dropout(dropout_p)

        # Fully connected layers
        # The input to the linear layer is num_filters because of global max
        ↪pooling
        self.fc1 = nn.Linear(num_filters, 50)
        self.fc2 = nn.Linear(50, num_classes)

    def forward(self, x):
        # Reshape input for Conv1d: (batch_size, channels, sequence_length)
        # Our features are the sequence, and we have 1 channel.
        x = x.unsqueeze(1)

        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)

        # Flatten the output for the fully connected layer
        x = x.view(x.size(0), -1)
        x = self.dropout(x)

        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

print("PyTorch 1D CNN model defined.")

```

PyTorch 1D CNN model defined.

```

[10]: def objective(trial):
    # --- Hyperparameter Search Space ---
    lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
    optimizer_name = trial.suggest_categorical("optimizer", ["Adam", "RMSprop",
    ↪"SGD"])
    num_filters = trial.suggest_int("num_filters", 16, 64, step=16)
    kernel_size = trial.suggest_int("kernel_size", 3, 7, step=2)
    dropout_p = trial.suggest_float("dropout_p", 0.1, 0.5)
    epochs = 15 # Fixed number of epochs for each trial for faster tuning

```



```

print(f"\n--- Starting Trial {trial.number} ---")
# --- K-Fold Cross-Validation ---
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
fold_accuracies = []

for fold, (train_idx, val_idx) in enumerate(kf.split(X_train_selected,
↪y_train)):
    # Data for this fold
    X_train_fold, X_val_fold = X_train_selected[train_idx],
↪X_train_selected[val_idx]
    y_train_fold, y_val_fold = y_train[train_idx], y_train[val_idx]

    # Convert to PyTorch Tensors, ensuring data is float32 to prevent errors
    X_train_tensor = torch.FloatTensor(X_train_fold.astype(np.float32))
    y_train_tensor = torch.LongTensor(y_train_fold)
    X_val_tensor = torch.FloatTensor(X_val_fold.astype(np.float32))
    y_val_tensor = torch.LongTensor(y_val_fold)

    # Create DataLoaders
    train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

    # --- Model Training and Evaluation for the Fold ---
    model = SingleShotCNN(INPUT_SIZE, num_classes=2,
↪num_filters=num_filters, kernel_size=kernel_size, dropout_p=dropout_p).
↪to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = getattr(optim, optimizer_name)(model.parameters(), lr=lr)

    # Training loop
    model.train()
    for epoch in range(epochs):
        print(f"    Epoch {epoch + 1}/{epochs}", end='\r')
        for batch_X, batch_y in train_loader:
            batch_X = batch_X.to(device)
            batch_y = batch_y.to(device)
            optimizer.zero_grad()
            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()
        print(f"    Fold {fold + 1} completed.")

    # Evaluation
    model.eval()
    with torch.no_grad():

```

```

        X_val_tensor = X_val_tensor.to(device)
        val_outputs = model(X_val_tensor)
        _, predicted = torch.max(val_outputs.data, 1)
        accuracy = accuracy_score(y_val_tensor.numpy(), predicted.cpu().
        ↪numpy())

        fold_accuracies.append(accuracy)
        print(f"    Fold Accuracy: {accuracy:.4f}")

    # Return the mean accuracy across all folds
    mean_accuracy = np.mean(fold_accuracies)
    print(f"--- Trial {trial.number} Finished | Mean Accuracy: {mean_accuracy:.
    ↪4f} ---")
    return np.mean(fold_accuracies)

print("Optuna objective function defined.")

```

Optuna objective function defined.

```

[11]: # Create and run the Optuna study
study = optuna.create_study(direction="maximize")
print("Starting Optuna hyperparameter search...")

# Increase n_trials for a more thorough search, e.g., 50 or 100
study.optimize(objective, n_trials=30)

print("\nOptuna study finished.")

# --- Display Results ---
best_trial = study.best_trial
print("\n--- Best Results ---")
print(f"Best Accuracy: {best_trial.value:.4f}")
print("Best Hyperparameters:")
for key, value in best_trial.params.items():
    print(f"    {key}: {value}")

# --- Final Evaluation ---
# Now, we train the final model on the FULL training data with the best_
↪hyperparameters
# and evaluate on the unseen test set.

print("\nTraining final model with best hyperparameters on all training data...
    ↪")
best_params = best_trial.params
final_model = SingleShotCNN(
    INPUT_SIZE,
    num_classes=2,
    num_filters=best_params['num_filters'],

```

```

        kernel_size=best_params['kernel_size'],
        dropout_p=best_params['dropout_p']
    ).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = getattr(optim, best_params['optimizer'])(final_model.parameters(),
    ↪lr=best_params['lr'])

# Create final dataloaders
X_train_tensor = torch.FloatTensor(X_train_selected)
y_train_tensor = torch.LongTensor(y_train)
X_test_tensor = torch.FloatTensor(X_test_selected)
y_test_tensor = torch.LongTensor(y_test)

final_train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
final_train_loader = DataLoader(final_train_dataset, batch_size=64,
    ↪shuffle=True)

# Final training loop
final_model.train()
for epoch in range(25): # Train for a few more epochs on the full data
    for batch_X, batch_y in final_train_loader:
        batch_X, batch_y = batch_X.to(device), batch_y.to(device)
        optimizer.zero_grad()
        outputs = final_model(batch_X)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()

# Final evaluation on the hold-out test set
final_model.eval()
with torch.no_grad():
    # 3. Move data tensor to the device
    test_outputs = final_model(X_test_tensor.to(device))
    _, predicted = torch.max(test_outputs.data, 1)
    # Move predicted tensor back to cpu for numpy conversion
# Convert ground truth and predicted labels to NumPy arrays
y_true = y_test_tensor.numpy()
y_pred = predicted.cpu().numpy()

# Calculate accuracy
final_accuracy = accuracy_score(y_true, y_pred)

# Generate classification report
report = classification_report(y_true, y_pred, target_names=["Class 0", "Class_
    ↪1"]) # Modify class names if needed

print("\n--- Final Model Performance ---")

```

```
print(f"Accuracy on unseen Test Set: {final_accuracy:.4f}")
print("\nClassification Report:")
print(report)
```

Starting Optuna hyperparameter search...

```
--- Starting Trial 0 ---
  Fold 1 completed.
  Fold Accuracy: 0.4971
  Fold 2 completed.
  Fold Accuracy: 0.5040
  Fold 3 completed.
  Fold Accuracy: 0.5528
  Fold 4 completed.
  Fold Accuracy: 0.5420
  Fold 5 completed.
  Fold Accuracy: 0.5364
--- Trial 0 Finished | Mean Accuracy: 0.5265 ---

--- Starting Trial 1 ---
  Fold 1 completed.
  Fold Accuracy: 0.5462
  Fold 2 completed.
  Fold Accuracy: 0.5765
  Fold 3 completed.
  Fold Accuracy: 0.5420
  Fold 4 completed.
  Fold Accuracy: 0.5423
  Fold 5 completed.
  Fold Accuracy: 0.5421
--- Trial 1 Finished | Mean Accuracy: 0.5498 ---

--- Starting Trial 2 ---
  Fold 1 completed.
  Fold Accuracy: 0.5420
  Fold 2 completed.
  Fold Accuracy: 0.5420
  Fold 3 completed.
  Fold Accuracy: 0.5420
  Fold 4 completed.
  Fold Accuracy: 0.5420
  Fold 5 completed.
  Fold Accuracy: 0.4579
--- Trial 2 Finished | Mean Accuracy: 0.5252 ---

--- Starting Trial 3 ---
  Fold 1 completed.
  Fold Accuracy: 0.5420
```

```
Fold 2 completed.
Fold Accuracy: 0.5420
Fold 3 completed.
Fold Accuracy: 0.5420
Fold 4 completed.
Fold Accuracy: 0.5420
Fold 5 completed.
Fold Accuracy: 0.5421
--- Trial 3 Finished | Mean Accuracy: 0.5420 ---

--- Starting Trial 4 ---
Fold 1 completed.
Fold Accuracy: 0.6839
Fold 2 completed.
Fold Accuracy: 0.5975
Fold 3 completed.
Fold Accuracy: 0.6540
Fold 4 completed.
Fold Accuracy: 0.6324
Fold 5 completed.
Fold Accuracy: 0.6565
--- Trial 4 Finished | Mean Accuracy: 0.6449 ---

--- Starting Trial 5 ---
Fold 1 completed.
Fold Accuracy: 0.8381
Fold 2 completed.
Fold Accuracy: 0.8268
Fold 3 completed.
Fold Accuracy: 0.8312
Fold 4 completed.
Fold Accuracy: 0.8287
Fold 5 completed.
Fold Accuracy: 0.8272
--- Trial 5 Finished | Mean Accuracy: 0.8304 ---

--- Starting Trial 6 ---
Fold 1 completed.
Fold Accuracy: 0.8459
Fold 2 completed.
Fold Accuracy: 0.8482
Fold 3 completed.
Fold Accuracy: 0.8393
Fold 4 completed.
Fold Accuracy: 0.8504
Fold 5 completed.
Fold Accuracy: 0.8424
--- Trial 6 Finished | Mean Accuracy: 0.8452 ---
```

```
--- Starting Trial 7 ---  
Fold 1 completed.  
Fold Accuracy: 0.5420  
Fold 2 completed.  
Fold Accuracy: 0.5420  
Fold 3 completed.  
Fold Accuracy: 0.5420  
Fold 4 completed.  
Fold Accuracy: 0.5420  
Fold 5 completed.  
Fold Accuracy: 0.5421  
--- Trial 7 Finished | Mean Accuracy: 0.5420 ---
```

```
--- Starting Trial 8 ---  
Fold 1 completed.  
Fold Accuracy: 0.7115  
Fold 2 completed.  
Fold Accuracy: 0.7111  
Fold 3 completed.  
Fold Accuracy: 0.7066  
Fold 4 completed.  
Fold Accuracy: 0.7072  
Fold 5 completed.  
Fold Accuracy: 0.7104  
--- Trial 8 Finished | Mean Accuracy: 0.7093 ---
```

```
--- Starting Trial 9 ---  
Fold 1 completed.  
Fold Accuracy: 0.6824  
Fold 2 completed.  
Fold Accuracy: 0.6751  
Fold 3 completed.  
Fold Accuracy: 0.6515  
Fold 4 completed.  
Fold Accuracy: 0.6207  
Fold 5 completed.  
Fold Accuracy: 0.6725  
--- Trial 9 Finished | Mean Accuracy: 0.6605 ---
```

```
--- Starting Trial 10 ---  
Fold 1 completed.  
Fold Accuracy: 0.6897  
Fold 2 completed.  
Fold Accuracy: 0.7291  
Epoch 1/15
```

[W 2025-07-02 14:48:34,349] Trial 10 failed with parameters: {'lr':

```
0.020077994456708426, 'optimizer': 'SGD', 'num_filters': 48, 'kernel_size': 7,
'dropout_p': 0.26500667707075576} because of the following error:
```

```
KeyboardInterrupt().
```

```
Traceback (most recent call last):
```

```
File "/opt/homebrew/Caskroom/miniconda/base/envs/aienv/lib/python3.10/site-
packages/optuna/study/_optimize.py", line 201, in _run_trial
```

```
    value_or_values = func(trial)
```

```
File "/var/folders/5d/z34b0t854mj6byb176t_pfgw0000gn/T/ipykernel_94449/3295628
962.py", line 40, in objective
```

```
    batch_X = batch_X.to(device)
```

```
KeyboardInterrupt
```

```
[W 2025-07-02 14:48:34,353] Trial 10 failed with value None.
```

```
Epoch 2/15
```

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[11], line 6
      3 print("Starting Optuna hyperparameter search...")
      5 # Increase n_trials for a more thorough search, e.g., 50 or 100
----> 6 study.optimize(objective, n_trials=30)
      8 print("\nOptuna study finished.")
     10 # --- Display Results ---

File /opt/homebrew/Caskroom/miniconda/base/envs/aienv/lib/python3.10/
site-packages/optuna/study/study.py:489, in Study.optimize(self, func,
n_trials, timeout, n_jobs, catch, callbacks, gc_after_trial, show_progress_bar)
     387 def optimize(
     388     self,
     389     func: ObjectiveFuncType,
     (...),
     396     show_progress_bar: bool = False,
     397 ) -> None:
     398     """Optimize an objective function.
     399
     400     Optimization is done by choosing a suitable set of hyperparameter_
     values from a given
     (...),
     487         If nested invocation of this method occurs.
     488     """
--> 489     _optimize(
     490         study=self,
     491         func=func,
     492         n_trials=n_trials,
     493         timeout=timeout,
     494         n_jobs=n_jobs,
     495         catch=tuple(catch) if isinstance(catch, Iterable) else (catch,)
     496         callbacks=callbacks,
```

```

497         gc_after_trial=gc_after_trial,
498         show_progress_bar=show_progress_bar,
499     )

```

File /opt/homebrew/Caskroom/miniconda/base/envs/aienv/lib/python3.10/site-packages/optuna/study/_optimize.py:64, in _optimize(study, func, n_trials, timeout, n_jobs, catch, callbacks, gc_after_trial, show_progress_bar)

```

62 try:
63     if n_jobs == 1:
---> 64         _optimize_sequential(
65             study,
66             func,
67             n_trials,
68             timeout,
69             catch,
70             callbacks,
71             gc_after_trial,
72             reseed_sampler_rng=False,
73             time_start=None,
74             progress_bar=progress_bar,
75         )
76     else:
77         if n_jobs == -1:

```

File /opt/homebrew/Caskroom/miniconda/base/envs/aienv/lib/python3.10/site-packages/optuna/study/_optimize.py:161, in _optimize_sequential(study, func, n_trials, timeout, catch, callbacks, gc_after_trial, reseed_sampler_rng, time_start, progress_bar)

```

158         break
160 try:
--> 161     frozen_trial = _run_trial(study, func, catch)
162 finally:
163     # The following line mitigates memory problems that can be occurred
↳in some
164     # environments (e.g., services that use computing containers such as
↳GitHub Actions).
165     # Please refer to the following PR for further details:
166     # https://github.com/optuna/optuna/pull/325.
167     if gc_after_trial:

```

File /opt/homebrew/Caskroom/miniconda/base/envs/aienv/lib/python3.10/site-packages/optuna/study/_optimize.py:253, in _run_trial(study, func, catch)

```

246     assert False, "Should not reach."
248 if (
249     frozen_trial.state == TrialState.FAIL
250     and func_err is not None
251     and not isinstance(func_err, catch)
252 ):

```



```

--> 253     raise func_err
      254 return frozen_trial

File /opt/homebrew/Caskroom/miniconda/base/envs/aienv/lib/python3.10/
site-packages/optuna/study/_optimize.py:201, in _run_trial(study, func, catch
199 with get_heartbeat_thread(trial._trial_id, study._storage):
200     try:
--> 201         value_or_values = func(trial)
202     except exceptions.TrialPruned as e:
203         # TODO(mamu): Handle multi-objective cases.
204         state = TrialState.PRUNED

Cell In[10], line 40, in objective(trial)
38 print(f"    Epoch {epoch + 1}/{epochs}", end='\r')
39 for batch_X, batch_y in train_loader:
---> 40     batch_X = batch_X.to(device)
41     batch_y = batch_y.to(device)
42     optimizer.zero_grad()

KeyboardInterrupt:

```

```

[14]: import logging

# --- Set up logging ---
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("training_log.log"), # Saves log to a file
        logging.StreamHandler()                  # Also prints to console
    ]
)

logger = logging.getLogger()

# --- Define Model ---
model = SingleShotCNN(
    input_size=X_train_selected.shape[1],
    num_classes=2,
    num_filters=64,
    kernel_size=7,
    dropout_p=0.20018027811185532
).to(device)

# --- Optimizer and Loss ---
optimizer = optim.Adam(model.parameters(), lr=0.002545768273574632)

```

```

criterion = nn.CrossEntropyLoss()

# --- Prepare Data ---
logger.info("Preparing data...")
X_train_tensor = torch.FloatTensor(X_train_selected).to(device)
y_train_tensor = torch.LongTensor(y_train).to(device)
X_test_tensor = torch.FloatTensor(X_test_selected).to(device)
y_test_tensor = torch.LongTensor(y_test)

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

# --- Training ---
logger.info("Starting training...")
model.train()
for epoch in range(25):
    running_loss = 0.0
    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()
        outputs = model(batch_X)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    logger.info(f"Epoch {epoch + 1}/25 - Loss: {running_loss:.4f}")

# --- Evaluation ---
logger.info("Evaluating on test set...")
model.eval()
with torch.no_grad():
    predictions = model(X_test_tensor)
    _, predicted_labels = torch.max(predictions, 1)

y_true = y_test_tensor.cpu().numpy()
y_pred = predicted_labels.cpu().numpy()

final_accuracy = accuracy_score(y_true, y_pred)
report = classification_report(y_true, y_pred, target_names=["Class 0", "Class_1"])

logger.info(f"Final Accuracy on Test Set: {final_accuracy:.4f}")
logger.info("Classification Report:\n" + report)

```

```

2025-07-02 14:50:41,736 [INFO] Preparing data...
2025-07-02 14:50:41,807 [INFO] Starting training...
2025-07-02 14:50:46,526 [INFO] Epoch 1/25 - Loss: 589.1349
2025-07-02 14:50:51,171 [INFO] Epoch 2/25 - Loss: 464.6132

```

```

2025-07-02 14:50:55,816 [INFO] Epoch 3/25 - Loss: 410.4984
2025-07-02 14:51:00,461 [INFO] Epoch 4/25 - Loss: 387.3099
2025-07-02 14:51:05,103 [INFO] Epoch 5/25 - Loss: 375.1918
2025-07-02 14:51:09,746 [INFO] Epoch 6/25 - Loss: 364.1503
2025-07-02 14:51:14,380 [INFO] Epoch 7/25 - Loss: 350.9186
2025-07-02 14:51:19,027 [INFO] Epoch 8/25 - Loss: 345.4506
2025-07-02 14:51:23,668 [INFO] Epoch 9/25 - Loss: 340.8469
2025-07-02 14:51:28,315 [INFO] Epoch 10/25 - Loss: 336.9296
2025-07-02 14:51:32,956 [INFO] Epoch 11/25 - Loss: 334.1317
2025-07-02 14:51:37,602 [INFO] Epoch 12/25 - Loss: 332.3263
2025-07-02 14:51:42,250 [INFO] Epoch 13/25 - Loss: 330.3940
2025-07-02 14:51:46,904 [INFO] Epoch 14/25 - Loss: 327.2787
2025-07-02 14:51:51,545 [INFO] Epoch 15/25 - Loss: 326.2947
2025-07-02 14:51:56,193 [INFO] Epoch 16/25 - Loss: 325.1216
2025-07-02 14:52:00,834 [INFO] Epoch 17/25 - Loss: 323.8918
2025-07-02 14:52:05,473 [INFO] Epoch 18/25 - Loss: 323.6132
2025-07-02 14:52:10,117 [INFO] Epoch 19/25 - Loss: 323.3124
2025-07-02 14:52:14,766 [INFO] Epoch 20/25 - Loss: 320.3815
2025-07-02 14:52:19,418 [INFO] Epoch 21/25 - Loss: 318.3337
2025-07-02 14:52:24,070 [INFO] Epoch 22/25 - Loss: 318.7121
2025-07-02 14:52:28,712 [INFO] Epoch 23/25 - Loss: 317.7154
2025-07-02 14:52:33,360 [INFO] Epoch 24/25 - Loss: 316.4594
2025-07-02 14:52:38,007 [INFO] Epoch 25/25 - Loss: 317.0025
2025-07-02 14:52:38,007 [INFO] Evaluating on test set...
2025-07-02 14:52:39,293 [INFO] Final Accuracy on Test Set: 0.8549
2025-07-02 14:52:39,295 [INFO] Classification Report:
      precision    recall  f1-score   support

   Class 0       0.87     0.86     0.87     8462
   Class 1       0.84     0.85     0.84     7148

 accuracy                   0.85     15610
 macro avg       0.85     0.85     0.85     15610
weighted avg       0.86     0.85     0.85     15610

```

```

[1]: # Save the best model to a .pkt file
best_model_path = "best_model.pkt"
torch.save(model.state_dict(), best_model_path)
print(f"Best model saved to {best_model_path}")

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[1], line 3
      1 # Save the best model to a .pkt file
      2 best_model_path = "best_model.pkt"
----> 3 torch.save(model.state_dict(), best_model_path)

```

```
4 print(f"Best model saved to {best_model_path}")
```

```
NameError: name 'torch' is not defined
```

```
[ ]:
```